Predictive Parsing (1)

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning outcomes

- After this lecture, you should be able to:
- write a “predictive parser” from a BNF grammar
- BNF
- define syntax
- write parser
- Parser
- code

Warmup: deriving strings from BNF

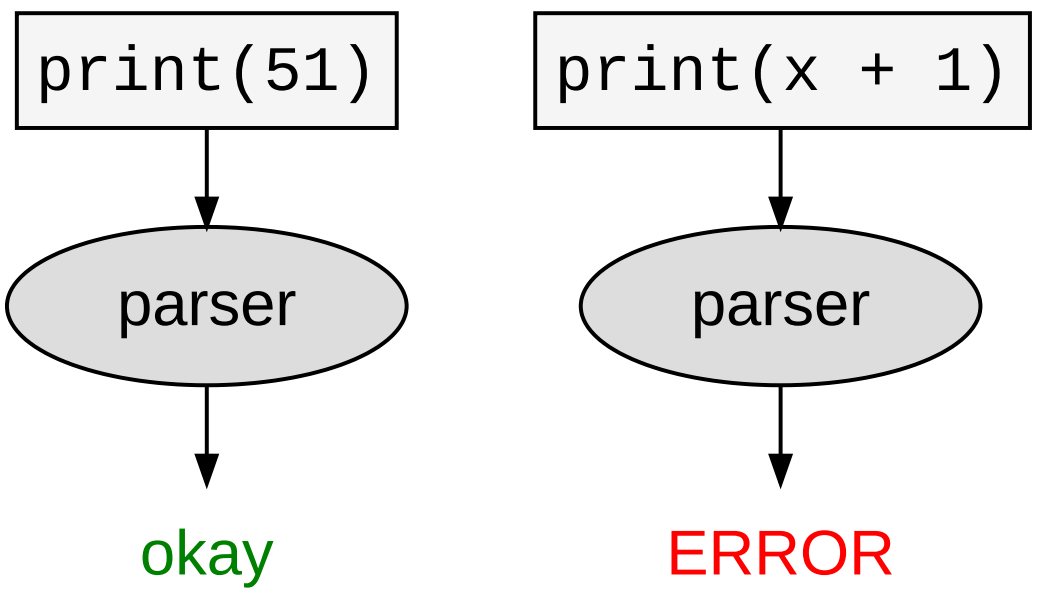
```
# mode: lhs+identifier
# terminals: print
stmt ::= print ( expr ) | id = expr
expr ::= id | num
num ::= [0-9]+
id ::= [a-zA-Z]+
```

Are these part of our language?

- `print(5)`
- `x = y`
- `x = print(5)`
- `print(x + 1)`

Recap: a parser checks syntax

```
1 # mode: lhs+identifier
2 # terminals: print
3 stmt ::= print ( expr ) | id = expr
4 expr ::= id | num
5 num ::= [0-9]+
6 id ::= [a-zA-Z]+
```



How to write parser for this grammar?

```
# mode: lhs+identifier
# terminals: print
stmt ::= print ( expr ) | id = expr
expr ::= id | num
num ::= [0-9]+
id ::= [a-zA-Z]+
```

Core Idea

- `stmt()` will parse statements
- `expr()` will parse expressions
- each function will decide which production to use by looking at the first symbol of each production
- once a production is picked the function will mimic the right-hand side of the production

Predictive parsing

- **Recursive-descent parsing**: “top-down method of syntax analysis in which we execute a set of recursive procedures to process the input” (Aho et al, Dragon book)
- **Predictive parsing**: “a form of recursive-descent parsing in which the lookahead symbol determines the procedure selected for each non-terminal” (Aho et al, Dragon book)

Main program

- initialize tokenizer
- call function associated with start symbol in grammar
- match on token DONE (end of input)

```
1 void parser() {
2   Tokenizer t = init__tokenizer(str_to_parse);
3   Stmt* e = parse_stmt(&t);
4   if (is_done(t)) { printf("Success!"); }
5 }
```

Example: function for ‘stmt’

```
# mode: lhs+identifier
# terminals: print
stmt ::= print ( expr ) | id = expr
expr ::= id | num
num ::= [0-9]+
id ::= [a-zA-Z]+
```

```
void stmt() {
  switch (token) {
    case PRINT:
      match(PRINT); match("("); expr(); match(")");
      break;
    case ID:
      match(ID); match("="); expr();
      break;
    default:
      error("syntax error");
  }
}
```

Challenge 1: Left recursion

- Any problems with a predictive parser here?

```
# terminals: term
# mode: lhs+identifier
R1 ::= R1 + term | term
```

The BNF can be transformed into this:

```
# mode: lhs+identifier
# terminals: term
R1 ::= term R2
R2 ::= + term R2 | ""
```

A general rule:

```
A ::= A α | β
```

Derives the same strings as:

```
A ::= β R
R ::= α R | ""
```

α, β are sequences of **terminals** and **non-terminals** that don't start with **A**

Exercise

Rewrite to eliminate left recursion

```
1 # mode: lhs+identifier
2 # terminals: var
3 R1 ::= R1 + R1 | var
```

Solution:

```
1 # mode: lhs+identifier
2 # terminals: var
3 R1 ::= var R2
4 R2 ::= + R1 R2 | ""
```

Another solution:

```
1 # mode: lhs+identifier
2 # terminals: var
3 R1 ::= var R2
4 R2 ::= + R1 R2 | ""
```

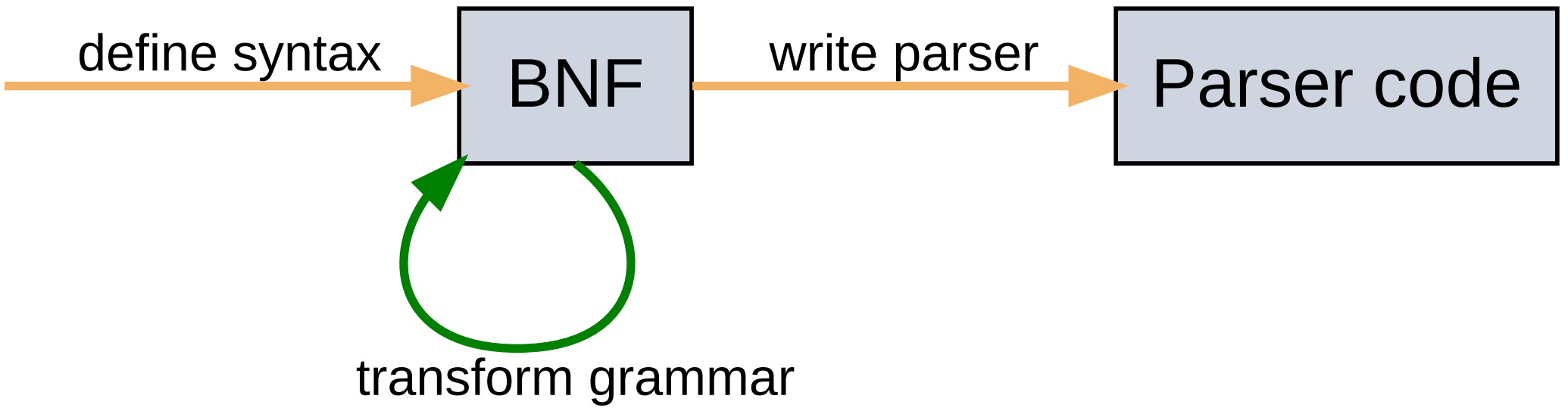
Challenge 2: Empty productions

```
# mode: lhs+identifier
# terminals: var
R1 ::= var R2
R2 ::= + R1 R2 | ""
```

How do we handle “” (i.e. the empty string, ϵ)?

```
void parse_R1() {
  switch (token) {
    case '+':
      match('+'); prase_R1(); prase_R2(); break;
    default:
      ;    // empty production
  }
}
```

Transforming a grammar



We transform the grammar in a way such that language of the grammar doesn’t change.

Summary

- predictive parsing is a simple kind of parser that is based directly on a BNF grammar
- but… you may need to modify a BNF grammar to allow predictive parsing to be used